

è un linguaggio finalizzato alla memorizzazione e all'interscambio di dati, non alla visualizzazione che, eventualmente, avviene applicando a un file XML regole di trasformazione basate sui fogli di stile CSS. Come risulta evidente dall'esempio precedente un corretto uso di XML – in particolare un'attenta scelta dei nomi dei *tag* – rende i documenti in questo formato, oltre che ideali per la gestione automatica da parte di applicazioni software, autodescrittivi per la lettura da parte dell'uomo, anche se il linguaggio è stato spesso criticato per la sua verbosità.

Dato che le regole per la nidificazione dei *tag* sono rigorosamente gerarchiche – come in HTML – i documenti in formato XML hanno naturalmente una struttura ad albero che viene utilizzata da molti strumenti che ne consentono la gestione.

ESEMPIO Il documento dell'esempio precedente presenta la struttura ad albero di FIGURA 1, che deriva dalla struttura di nidificazione dei *tag* con l'elemento che racchiude tutti gli altri e che diviene il nodo radice dell'albero.

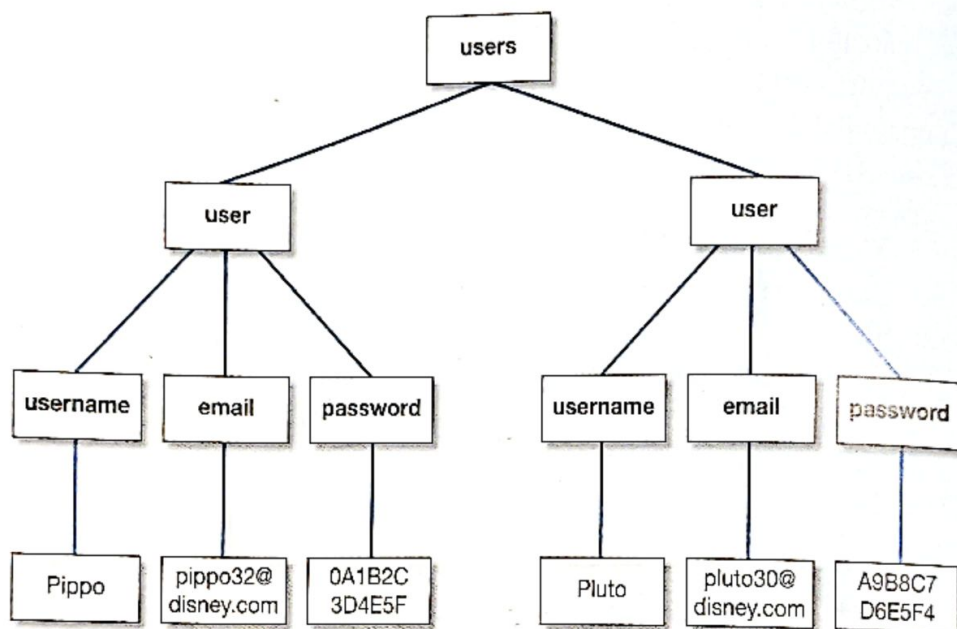


FIGURA 1

1 La sintassi del linguaggio XML e la struttura ad albero dei documenti

Le regole di base della sintassi XML non sono complesse:

- tutti i documenti XML devono iniziare con la seguente dichiarazione che identifica la versione di riferimento del linguaggio
`<?xml version="1.0" ?>`
- i nomi dei *tag* possono iniziare esclusivamente con un carattere alfabetico o con il simbolo «_»;

- i nomi dei *tag* possono contenere esclusivamente caratteri alfabetici, cifre numeriche e i simboli «_», «-» e «.» (sono quindi esclusi gli spazi);
- i nomi dei *tag* sono sensibili alla differenza tra caratteri minuscoli e maiuscoli;
- tutti i *tag* devono essere chiusi con un *tag* corrispondente avente lo stesso nome preceduto dal simbolo «/»;
- la nidificazione dei *tag* aperti/chiusi deve essere rigorosamente gerarchica;
- l'intero documento XML, a esclusione della dichiarazione iniziale, deve essere compreso in un solo elemento «radice», cioè tra due *tag* accoppiati di apertura e chiusura dell'elemento;
- i *tag* possono avere attributi per i quali è possibile specificare valori che devono essere necessariamente compresi tra due simboli «"» o «'»;
- i simboli chiave del linguaggio sono rappresentati mediante le seguenti entità (TABELLA 1)

TABELLA 1

<	<
>	>
&	&
"	"
'	'

- un commento è compreso tra i simboli «<!--» e «-->».

ESEMPIO Il seguente file di testo è un documento XML corretto che rappresenta un messaggio:

```
<?xml version="1.0" ?>
<MESSAGE timestamp="07/10/2021 12:00:00">
  <from>Giorgio</from>
  <to>Fiorenzo</to>
  <text>C'è erano molti invitati al tuo compleanno oggi?</text>
</MESSAGE>
```

L'elemento radice è MESSAGE e timestamp è un suo attributo.

Un **elemento XML** è quanto è compreso tra un *tag* di apertura e il corrispondente *tag* di chiusura (inclusi); un elemento può contenere testo, attributi e altri elementi, oppure essere vuoto.

L'elemento MESSAGGIO dell'esempio precedente comprende l'attributo timestamp e gli elementi from, to e text che contengono esclusivamente testo.

Esiste una sintassi abbreviata per gli elementi vuoti, o che comprendono solo attributi, che impiega un solo *tag* che contemporaneamente apre e chiude l'elemento stesso, come nei seguenti esempi:

<tag/>

<tag attributo="..." />

Il linguaggio XML prevede un attributo predefinito speciale denominato `xml:lang` per indicare la lingua del testo contenuto in un elemento; i valori più comuni sono i seguenti (TABELLA 2):

TABELLA 2

en	Inglese
fr	Francese
de	Tedesco
es	Spagnolo
it	Italiano

ESEMPIO Il documento XML dell'esempio precedente dovrebbe riportare l'attributo per l'indicazione del linguaggio:

```
<?xml version="1.0" ?>
<MESSAGE timestamp="07/10/2021 12:00:00">
  <from>Giorgio</from>
  <to>Fiorenzo</to>
  <text xml:lang="it">
    C'&apost;erano molti invitati al tuo compleanno oggi?
  </text>
</MESSAGE>
```

Dato che un elemento può contenere a sua volta altri elementi, gli strumenti software che gestiscono i documenti XML analizzano l'intero contenuto di ogni elemento. Per evitare che il contenuto di un elemento sia analizzato e permettere che al suo interno possa essere inserita qualsiasi sequenza di caratteri, è possibile qualificarlo come `CDATA` (*Character DATA*) inserendolo tra i simboli `<<![CDATA[` e `>>]]>`.

ESEMPIO Il seguente file di testo è un documento XML corretto che rappresenta un messaggio:

```
<?xml version="1.0" ?>
<MESSAGE timestamp="07/10/2021 12:00:00">
  <from>Giorgio</from>
  <to>Fiorenzo</to>
  <text>
    <![CDATA[
      Ti invio un esempio di XML:
      <?xml version="1.0" ?>
      <users>
        <user>
          <username>Pippo</username>
          <email>pippo32@disney.com</email>
        </user>
        <user>
          <username>Pluto</username>
          <email>pluto30@disney.com</email>
        </user>
      </users>
    ]]>
  </text>
</MESSAGE>
```

OSSERVAZIONE La possibilità di definire testo di tipo CDATA consente per esempio di inserire codice HTML come contenuto di un elemento di un file XML.

Se il file che contiene un documento XML contiene caratteri che non appartengono all'insieme ASCII, come per esempio i caratteri accentati della nostra lingua, è necessario specificare il tipo di codifica utilizzato di cui la TABELLA 3 riporta i più comuni².

TABELLA 3

ISO-8859-1	Codifica standard a 8 bit dei caratteri dell'Europa occidentale.
UTF-8	Codifica Unicode che utilizza da 8 a 32 bit per il singolo carattere.
UTF-16	Codifica Unicode che utilizza 16 bit per il singolo carattere ed è adottata dal linguaggio Java.

La codifica dei caratteri viene specificata come attributo della dichiarazione XML iniziale:

```
<?xml version="1.0" encoding="..." ?>
```

ESEMPIO

Il seguente file di testo è un documento XML corretto che rappresenta un messaggio:

```
<?xml version="1.0" encoding="ISO-8859-1" ?>
<MESSAGE timestamp="07/10/2021 12:00:00">
  <from>Giorgio</from>
  <to>Fiorenzo</to>
  <text>Perché non festeggi il tuo compleanno oggi?</text>
</MESSAGE>
```

In XML gli attributi dovrebbero essere utilizzati esclusivamente per fornire informazioni accessorie e non per rappresentare i dati.

ESEMPIO

Se la data/ora di ricezione di un messaggio è un dato che lo caratterizza, l'esempio precedente dovrebbe essere riscritto come segue, trasformando l'attributo timestamp negli elementi date e time:

```
<?xml version="1.0" encoding="ISO-8859-1" ?>
<MESSAGE>
  <date>07/10/2021</date>
  <time>12:00:00</time>
  <from>Giorgio</from>
  <to>Fiorenzo</to>
  <text>Perché non festeggi il tuo compleanno oggi?</text>
</MESSAGE>
```

È sempre possibile rappresentare la struttura di un documento XML sintatticamente corretto mediante un diagramma ad albero in cui il nodo radice è l'elemento radice del documento e i nodi dell'albero sono gli elementi stessi, gli attributi degli elementi, il contenuto testuale degli elementi e gli eventuali commenti inseriti nel documento.

2. Il formato in cui viene salvato il file deve essere coerente con il tipo di codifica specificato: singolo byte per ASCII, UTF-8 e ISO-8859-1, multibyte per UTF-16.

La visualizzazione di un documento XML con XSLT

I browser, gli editor avanzati e gli ambienti software per gli sviluppatori visualizzano un file contenente un documento XML in formato testuale fornendo strumenti per nascondere il contenuto di un elemento e, di conseguenza, facilitare la lettura della gerarchia degli elementi del documento stesso.

Volendo realizzare una pagina web che visualizzi il contenuto di un documento XML in modo appropriato è necessario specificare un documento in linguaggio XSL (*eXtensible Stylesheet Language*).

XSL comprende XSLT (*eXtensible Stylesheet Language Transformations*), che consente di specificare regole per la trasformazione di un documento XML in un diverso documento XML, o in una pagina HTML.

Il seguente file di testo è un documento XML corretto che rappresenta un libro:

```
<?xml version="1.0" encoding="ISO-8859-1" ?>
<libro genere="fantascienza" xml:lang="it">
  <autore>Stefano Benni</autore>
  <titolo>Terra!</titolo>
  <editore>Feltrinelli</editore>
  <anno>1983</anno>
</libro>
```

Esso è rappresentato dal diagramma ad albero di FIGURA 2.

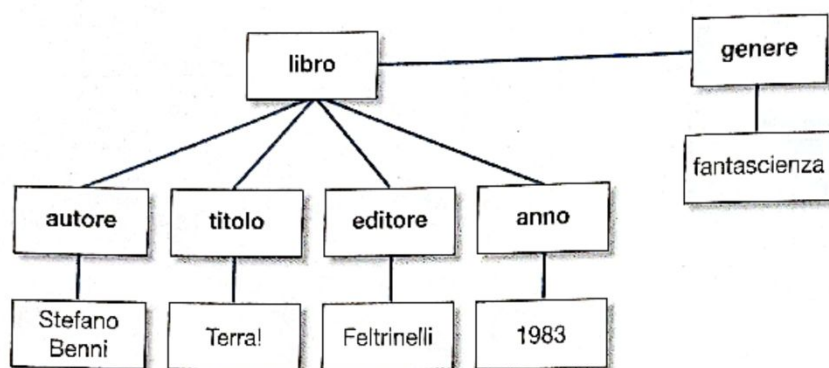


FIGURA 2

2 La definizione di linguaggi XML mediante schemi XSD

Un documento XML che rispetta le regole sintattiche di base è denominato **ben formato**.

OSSERVAZIONE La verifica del rispetto delle regole sintattiche di un documento XML può essere effettuata utilizzando specifici strumenti software: molti browser o editor specializzati per programmatori sono in grado di verificare che un documento XML sia ben formato.

Dato che XML viene utilizzato per definire nuovi linguaggi specifici che utilizzano *tag* marcatori non predefiniti, anche se un documento XML è ben formato rimane aperto il problema di verificare che impieghi effettivamente i *tag* previsti e in modo corretto; inoltre il contenuto di un elemento rappresenta spesso un dato che deve rispettare regole di tipizzazione.

Il seguente documento XML che rappresenta una collezione di libri è ben formato:

```
<?xml version="1.0" ?>
<libri>
  <libro genere="fantascienza">
    <autore>Stefano Benni</autore>
    <titolo>Terra!</titolo>
    <editore>Feltrinelli</editore>
```

Linguaggi XML

Il successo di XML è legato alla sua flessibilità: spesso le aziende e gli sviluppatori software definiscono schemi XML per i file che costituiscono l'input o l'output delle applicazioni che realizzano. Per esempio, i formati dei file prodotti dalle applicazioni Office di Microsoft sono in formato XML (da cui deriva il carattere «x» che ne termina l'estensione del nome).

In alcuni casi sono stati definiti dei veri e propri linguaggi XML, aventi scopi specifici; tra questi ricordiamo: **CML** (Chemical Markup Language), che è usato per la definizione di strutture molecolari; **EPUB** (Electronic Publication), che è un formato aperto per la pubblicazione di e-book; **GML** (Geographic Markup Language) e **KML** (Keyhole Markup Language), che sono linguaggi per la definizione di risorse georeferenziate; **MathML** (Mathematical Markup Language), che è impiegato per la codifica della notazione matematica; **ODF** (Open Document Format), che è un formato aperto per i documenti delle applicazioni di utilità; **SVG** (Scalable Vector Graphics), che è un formato utilizzato per la grafica vettoriale bidimensionale.

```

    <anno>83</anno>
  </libro>
  <libro genere="romanzo">
    <scrittore>Mark Twain</scrittore>
    <titolo>Le avventure di Huckleberry Finn</titolo>
    <anno>1884</anno>
  </libro>
  <libro>
    <autore>Herman Melville</autore>
    <titolo>Moby Dick</titolo>
    <editore/>
    <anno>milleottocentocinquantuno</anno>
  </libro>
</libri>

```

Tuttavia i dati relativi ai singoli libri non sono presenti in modo coerente: per «Moby Dick» l'anno di pubblicazione non è espresso in formato numerico e l'editore non è indicato; per il capolavoro di Mark Twain non è specificato un elemento *editore* e l'autore è definito come contenuto dell'elemento *scrittore*; infine l'anno di pubblicazione di «Terra!» è indicato in forma numerica abbreviata.

Un file di questo tipo risulterebbe di difficile elaborazione da parte di un'applicazione software.

L'esempio precedente evidenzia la necessità di un metodo per definire la struttura che un documento XML relativo a una specifica applicazione deve avere e la tipologia che i dati contenuti negli elementi devono rispettare. Il primo formato standard per la definizione della struttura di un documento XML è stato DTD, *Document Type Definition*.

DTD presentava molte limitazioni – tra cui il fatto di non avere una sintassi compatibile con XML – che sono state superate da XSD (*XML Schema Definition*). Il metodo più comune per definire uno schema di validazione per un documento XML è XSD, inizialmente proposto da W3C nel 2001³.

Uno schema XSD definisce:

- gli elementi di un documento XML e la loro relazione gerarchica;
- gli eventuali attributi degli elementi;
- il numero e l'ordinamento degli elementi;
- gli eventuali elementi vuoti;
- il tipo dei dati contenuti negli elementi e negli attributi;
- i valori predefiniti o costanti del contenuto degli elementi e degli attributi.

Il linguaggio XSD che consente di definire lo schema di un documento XML è a sua volta un linguaggio XML: ogni schema XSD deve avere come radice un elemento denominato **schema** con un attributo predefinito che riferisce il file contenente gli elementi e i tipi di dati standard utilizzabili:

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
```

3. La versione 1.1 di XSD è divenuta una raccomandazione W3C nel 2012.

Inoltre ogni documento XML dovrebbe dichiarare il file contenente lo schema di riferimento inserendo i seguenti attributi predefiniti nella definizione dell'elemento radice:

```
xmlns:xsi="http://www.w3.org/XMLSchema-instance"
xsi:noNamespaceSchemaLocation="..."
```

OSSERVAZIONE Essendo XML una tecnologia molto diffusa nelle applicazioni web, spesso il file dello schema di riferimento è una risorsa esposta in rete e definita da un URL.

RELAX NG

REgular Language for XML Next Generation è un linguaggio per definire la struttura e il contenuto di documenti XML alternativo a XSD rispetto al quale è considerato più semplice. Nella sua forma base uno schema RELAX NG è un documento XML, ma ne esiste anche una forma compatta che condivide molte funzionalità degli schemi DTD.

ESEMPIO

Un possibile schema XSD per un documento XML che rappresenta un singolo libro potrebbe essere il seguente:

```
<?xml version="1.0" ?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="libro">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="autore" type="xs:string"/>
        <xs:element name="titolo" type="xs:string"/>
        <xs:element name="editore" type="xs:string"/>
        <xs:element name="anno" type="xs:integer"/>
      </xs:sequence>
    </xs:complexType>
  </xs:element>
</xs:schema>
```

Lo schema definisce un documento XML che deve avere un elemento radice `libro` con un attributo `genre` di tipo stringa di caratteri e gli elementi `autore`, `titolo` ed `editore` di tipo stringa di caratteri più l'elemento `anno` di tipo numerico intero.

Un documento XML deve riferire lo schema a cui si attiene nella definizione dell'elemento radice, come nel seguente documento XML, in cui si assume che lo schema precedente sia stato salvato nel file `libro.xsd`:

```
<?xml version="1.0" ?>
<libro
  xmlns:xsi="http://www.w3.org/XMLSchema-instance"
  xsi:noNamespaceSchemaLocation="libro.xsd">
  <autore>Stefano Benni</autore>
  <titolo>Terra!</titolo>
  <editore>Feltrinelli</editore>
  <anno>1983</anno>
</libro>
```

Progettare un linguaggio XML significa definirne lo schema, cioè la sintassi che un qualsiasi documento XML deve rispettare per appartenere al linguaggio.

Un documento XML **valido** è un documento XML ben formato che rispetta le regole dello schema che riferisce.

La validazione di un documento XML rispetto allo schema riferito viene normalmente effettuata mediante strumenti software automatici.

Validazione di documenti XML con Notepad++

Notepad++ (<http://notepad-plus-plus.org>) è un editor per sviluppatori che utilizzano il sistema operativo Windows, gratuito e molto noto.

Tra i numerosi *plug-in* di cui dispone Notepad++, *XML tools* consente di verificare che un file in formato XML sia ben formato e di validare un file in formato XML rispetto a uno schema XSD.

2.1 La definizione degli elementi semplici e degli attributi

Un **elemento semplice** è un elemento XML che può contenere solo testo: non può includere attributi o altri elementi. Dato che XML è un formato testuale, il testo può rappresentare stringhe di caratteri, oppure valori numerici, date, orari, ...

In uno schema XSD è possibile imporre un tipo al testo contenuto in un elemento.

La sintassi per definire un elemento semplice in uno schema XSD è la seguente:

```
<xs:element name="..." type="..."/>
```

I tipi più comuni per un elemento semplice sono riportati nella TABELLA 4:

TABELLA 4

xs:string	Stringhe di caratteri.
xs:decimal	Valori numerici non necessariamente interi.
xs:integer	Valori numerici interi.
xs:date	Data nel formato aaaa-mm-gg (a: anno, m: mese, g: giorno).
xs:time	Ora nel formato oo-mm-ss (o: ora, m: minuti, s: secondi).

ESEMPI

Esempi di definizione di elementi semplici

```
<xs:element name="nome" type="xs:string"/>
<xs:element name="dataNascita" type="xs:date"/>
<xs:element name="età" type="xs:integer"/>
```

e di relativi elementi XML validi

```
<nome>Walt Disney</nome>
<dataNascita>1901-12-05</dataNascita>
<età>65</età>
```

e invalidi:

```
<nome/>
<dataNascita>5/12/1901</dataNascita>
<età>sessantacinque</età>
```

Un elemento semplice può avere un valore predefinito che assume se il documento XML non ne specifica uno, o un valore costante che impedisce che nel documento XML ne possa essere definito uno diverso: per la definizione si utilizzano rispettivamente gli attributi `default` e `fixed`.

ESEMPI

Esempi di definizione di elementi semplici con valori predefiniti e costanti:

```
<xs:element name="colore" type="xs:string" default="rosso"/>
<xs:element name="piGreco" type="xs:decimal" fixed="3.14159"/>
```


e di relativi elementi XML validi:

```
<colore>verde</colore>
<colore/>
<piGreco>3.14159</piGreco>
```

e invalidi:

```
<piGreco>3.1416</piGreco>
```

Nella definizione di un elemento semplice è possibile impostare delle restrizioni al tipo base indicato. Per i tipi numerici sono possibili le restrizioni indicate nella TABELLA 5.

TABELLA 5

xs:fractionDigits	Massimo numero di cifre decimali.
xs:maxExclusive xs:maxInclusive	Valore massimo (rispettivamente escluso e incluso).
xs:minExclusive xs:minInclusive	Valore minimo (rispettivamente escluso e incluso).
xs:totalDigits	Numero esatto di cifre.

L'impostazione di restrizioni a un tipo di base impone l'uso della notazione estesa per definire un elemento semplice:

```
<xs:element name="...">
  <xs:simpleType>
    <xs:restriction base="xs:...">
      ...
      <!--restrizioni -->
      ...
    </xs:restriction>
  </xs:simpleType>
</xs:element>
```

ESEMPI

Esempi di definizione di elementi semplici con restrizioni:

```
<xs:element name="quantità">
  <xs:simpleType>
    <xs:restriction base="xs:integer">
      <xs:minExclusive value="0"/>
      <xs:maxInclusive value="10"/>
    </xs:restriction>
  </xs:simpleType>
</xs:element>
```

```
<xs:element name="prezzo">
  <xs:simpleType>
    <xs:restriction base="xs:decimal">
      <xs:fractionDigits value="2"/>
    </xs:restriction>
  </xs:simpleType>
</xs:element>
```

e di relativi elementi XML validi

```
<quantità>5</quantità>
```

Le espressioni regolari e la definizione dei pattern

Una possibile restrizione per il tipo stringa di caratteri consiste nello specificare un *pattern* cui la stringa deve corrispondere. Per esempio, un codice fiscale deve essere composto nell'ordine da 6 caratteri alfabetici maiuscoli, 2 cifre numeriche, 1 carattere alfabetico maiuscolo, 2 cifre numeriche, 1 carattere alfabetico maiuscolo, 3 cifre numeriche e un carattere alfabetico maiuscolo finale.

Il linguaggio XSD consente di definire questo tipo di condizioni mediante la restrizione `xs:pattern` e un valore che rappresenta un'espressione regolare standard.

e invalidi:

```
<quantità>10</quantità>
<prezzo>0.99</prezzo>
<prezzo>99</prezzo>

<quantità>0</quantità>
<quantità>100</quantità>
<prezzo>1.999</prezzo>
```

Per le stringhe di caratteri sono possibili le restrizioni indicate nella TABELLA 6.

TABELLA 6

xs:length	Numero esatto di caratteri.
xs:maxLength	Numero massimo di caratteri.
xs:minLength	Numero minimo di caratteri.

ESEMPIO

Esempi di definizione di elementi semplici con restrizioni:

```
<xs:element name="indirizzo">
  <xs:simpleType>
    <xs:restriction base="xs:string">
      <xs:minLength value="10"/>
      <xs:maxLength value="100"/>
    </xs:restriction>
  </xs:simpleType>
</xs:element>

<xs:element name="CAP">
  <xs:simpleType>
    <xs:restriction base="xs:string">
      <xs:length value="5"/>
    </xs:restriction>
  </xs:simpleType>
</xs:element>
```

e di relativi elementi XML validi

```
<indirizzo>Via della Venezia 1, Livorno</indirizzo>
<CAP>57100</CAP>
```

e invalidi:

```
<indirizzo>Via corta</indirizzo>
<CAP>1234</CAP>
```

Una restrizione molto comune del tipo di base stringa di caratteri – l'enumerazione – consente di elencare le stringhe valide.

ESEMPIO

L'elemento colore può contenere una sola tra le stringhe: bianco, nero, rosso, verde, blu, celeste, violetto e giallo:

```
<xs:element name="colore">
  <xs:simpleType>
    <xs:restriction base="xs:string">
      <xs:enumeration value="bianco"/>
```



```

<xs:enumeration value="nero"/>
<xs:enumeration value="rosso"/>
<xs:enumeration value="verde"/>
<xs:enumeration value="blu"/>
<xs:enumeration value="celeste"/>
<xs:enumeration value="violetto"/>
<xs:enumeration value="giallo"/>
</xs:restriction>
</xs:simpleType>
</xs:element>

```

I seguenti sono elementi XML validi

```

<colore>nero</colore>
<colore>giallo</colore>

```

e invalidi:

```

<colore>grigio</colore>
<colore/>

```

Un tipo enumerativo, così come qualsiasi altro tipo, può essere definito autonomamente e successivamente utilizzato per definire uno o più elementi. L'esempio precedente può essere così riscritto:

```

<xs:simpleType name="nomeColore">
  <xs:restriction base="xs:string">
    <xs:enumeration value="bianco"/>
    <xs:enumeration value="nero"/>
    <xs:enumeration value="rosso"/>
    <xs:enumeration value="verde"/>
    <xs:enumeration value="blu"/>
    <xs:enumeration value="celeste"/>
    <xs:enumeration value="violetto"/>
    <xs:enumeration value="giallo"/>
  </xs:restriction>
</xs:simpleType>

<xs:element name="colore" type="nomeColore"/>

```

Un elemento semplice non può prevedere attributi, ma in uno schema XSD un attributo viene definito in modo analogo a un elemento semplice:

```

<xs:attribute name="..." type="..."/>

```

Un attributo è normalmente facoltativo; per renderlo obbligatorio nella definizione deve essere valorizzato con `required` l'attributo `use`:

```

<xs:attribute name="..." type="..." use="required"/>

```

2.2 La definizione degli elementi complessi

Un **elemento complesso** è un elemento XML che ricade in una delle seguenti categorie:

- è un elemento vuoto, eventualmente con attributi;

- è un elemento che contiene altri elementi;
- è un elemento che contiene solo testo, ma con attributi;
- è un elemento che contiene sia testo sia altri elementi.

Un elemento complesso può essere definito direttamente, ma spesso viene definito mediante un **tipo** che permette di definire più elementi complessi.

ESEMPIO La definizione degli elementi docente e studente

```
<xs:element name="docente">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="cognome" type="xs:string"/>
      <xs:element name="nome" type="xs:string"/>
      <xs:element name="dataNascita" type="xs:date"/>
      <xs:element name="luogoNascita" type="xs:string"/>
    </xs:sequence>
  </xs:complexType>
</xs:element>

<xs:element name="studente">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="cognome" type="xs:string"/>
      <xs:element name="nome" type="xs:string"/>
      <xs:element name="dataNascita" type="xs:date"/>
      <xs:element name="luogoNascita" type="xs:string"/>
    </xs:sequence>
  </xs:complexType>
</xs:element>
```

può essere riscritta utilizzando il seguente tipo persona:

```
<xs:complexType name="persona">
  <xs:sequence>
    <xs:element name="cognome" type="xs:string"/>
    <xs:element name="nome" type="xs:string"/>
    <xs:element name="dataNascita" type="xs:date"/>
    <xs:element name="luogoNascita" type="xs:string"/>
  </xs:sequence>
</xs:complexType>

<xs:element name="docente" type="persona"/>
<xs:element name="studente" type="persona"/>
```

L'indicatore `<xs:sequence>`, utilizzato nello schema XSD precedente, impone che gli elementi complessi docente e studente contengano ciascuno gli elementi semplici *cognome*, *nome*, *dataNascita* e *luogoNascita* nell'ordine specificato, come nel seguente esempio:

```
<docente>
  <cognome>Meini</cognome>
  <nome>Giorgio</nome>
  <dataNascita>1965-03-23</dataNascita>
  <luogoNascita>Livorno</luogoNascita>
</docente>
```


Un elemento vuoto è un elemento complesso così definito

```
<xs:element name="elementoVuoto">
  <xs:complexType>
  </xs:complexType>
</xs:element>
```

per il quale sono validi elementi XML come il seguente:

```
<elementoVuoto></elementoVuoto>
<elementoVuoto/>
```

Un elemento vuoto con un attributo è invece un elemento complesso così definito

```
<xs:element name="elementoVuoto">
  <xs:complexType>
    <xs:attribute name="attributo" type="xs:string"/>
  </xs:complexType>
</xs:element>
```

per il quale sono validi elementi XML come il seguente:

```
<elementoVuoto></elementoVuoto>
<elementoVuoto attributo="valore"/>
```

Il modo più comune per definire un elemento complesso che contiene esclusivamente altri elementi è il ricorso all'indicatore `<xs:sequence>` che nello schema XSD comprende l'elenco ordinato degli elementi contenuti.

ESEMPIO

La seguente definizione dell'elemento indirizzo

```
<xs:element name="indirizzo">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="via" type="xs:string"/>
      <xs:element name="numero">
        <xs:simpleType>
          <xs:restriction base="xs:integer">
            <xs:minInclusive value="1"/>
          </xs:restriction>
        </xs:simpleType>
      </xs:element>
      <xs:element name="CAP">
        <xs:simpleType>
          <xs:restriction base="xs:string">
            <xs:length value="5"/>
          </xs:restriction>
        </xs:simpleType>
      </xs:element>
      <xs:element name="città" type="xs:string"/>
    </xs:sequence>
  </xs:complexType>
</xs:element>
```

valida elementi XML come questo

```
<indirizzo>
  <via>Via della Venezia</via>
  <numero>1</numero>
  <CAP>57100</CAP>
  <città>Livorno</città>
</indirizzo>
```

ma non come quello che segue

```
<indirizzo>
  <via>Via della Venezia</via>
  <numero>1</numero>
  <città>Livorno</città>
  <CAP>57100</CAP>
</indirizzo>
```

in quanto l'indicatore `<xs:sequence>` vincola l'ordine degli elementi contenuti nell'elemento complesso indirizzo.

Un elemento XML che contiene solo testo è semplice, ma un elemento che contiene solo testo e prevede uno o più attributi è un elemento complesso con contenuto semplice. Elementi complessi di questo tipo possono essere definiti ricorrendo a un'estensione di un tipo base:

```
<xs:element name="elementoConAttributo">
  <xs:complexType>
    <xs:simpleContent>
      <xs:extension base="...">
        <attribute name="..." type="..." />
      </xs:extension>
    </xs:simpleContent>
  </xs:complexType>
</xs:element>
```

ESEMPIO La seguente definizione dell'elemento misura

```
<xs:element name="misura">
  <xs:complexType>
    <xs:simpleContent>
      <xs:extension base="xs:decimal">
        <xs:attribute name="unità" type="xs:string" use="required" />
      </xs:extension>
    </xs:simpleContent>
  </xs:complexType>
</xs:element>
```

valida elementi XML come questi

```
<misura unità="cm">100</misura>
<misura unità="mm">0.001</misura>
```

ma non come quello che segue

```
<misura>10</misura>
```

in quanto l'attributo `use` impostato a `required` nello schema rende la presenza dell'attributo `unità` obbligatoria.

Per definire un elemento XML che possa contenere sia testo sia elementi è necessario impostare l'attributo `mixed` a `true` nella definizione di un tipo complesso.

ESEMPIO La seguente definizione dell'elemento `messaggio`

```
<xs:element name="messaggio">
  <xs:complexType mixed="true">
    <xs:sequence>
      <xs:element name="codice" type="xs:string"/>
      <xs:element name="data" type="xs:date"/>
      <xs:element name="ora" type="xs:time"/>
      <xs:element name="luogo" type="xs:string"/>
    </xs:sequence>
  </xs:complexType>
</xs:element>
```

permette di validare elementi XML come quello che segue:

```
<messaggio>
La spedizione <codice>0123456789</codice> è stata consegnata il giorno
<data>2021-10-22</data> alle ore <ora>15:30</ora> a <luogo>Livorno</luogo>
</messaggio>
```

Oltre all'indicatore `<xs:sequence>` ne esistono altri due per definire gli elementi contenuti in un elemento complesso:

- `<xs:all>` è analogo a `<xs:sequence>`, ma non impone il rispetto dell'ordine degli elementi;
- `<xs:choice>` permette di elencare due o più elementi di cui uno solo deve necessariamente essere presente.

ESEMPIO La seguente definizione dell'elemento `indirizzo`

```
<xs:element name="indirizzo">
  <xs:complexType>
    <xs:all>
      <xs:element name="via" type="xs:string"/>
      <xs:element name="numero">
        <xs:simpleType>
          <xs:restriction base="xs:integer">
            <xs:minInclusive value="1"/>
          </xs:restriction>
        </xs:simpleType>
      </xs:element>
      <xs:element name="CAP">
        <xs:simpleType>
          <xs:restriction base="xs:string">
            <xs:length value="5"/>
          </xs:restriction>
        </xs:simpleType>
      </xs:element>
      <xs:element name="città" type="xs:string"/>
    </xs:all>
  </xs:complexType>
</xs:element>
```

valida entrambi i seguenti elementi XML

```
<indirizzo>
  <via>Via della Venezia</via>
  <numero>1</numero>
  <CAP>57100</CAP>
  <città>Livorno</città>
</indirizzo>
```

```
<indirizzo>
  <via>Via della Venezia</via>
  <numero>1</numero>
  <città>Livorno</città>
  <CAP>57100</CAP>
</indirizzo>
```

in quanto l'indicatore `<xs:all>` non vincola l'ordine degli elementi contenuti nell'elemento complesso `indirizzo`.

ESEMPIO

Data la seguente definizione dell'elemento complesso `referimentoCliente`

```
<xs:element name="referimentoCliente">
  <xs:complexType>
    <xs:choice>
      <xs:element name="codiceFiscale">
        <xs:simpleType>
          <xs:restriction base="xs:string">
            <xs:length value="16"/>
          </xs:restriction>
        </xs:simpleType>
      </xs:element>
      <xs:element name="partitaIVA">
        <xs:simpleType>
          <xs:restriction base="xs:string">
            <xs:length value="11"/>
          </xs:restriction>
        </xs:simpleType>
      </xs:element>
    </xs:choice>
  </xs:complexType>
</xs:element>
```

sono validi i seguenti elementi XML

```
<referimentoCliente>
  <codiceFiscale>MNEGRG65C23E625Y</codiceFiscale>
</referimentoCliente>
```

```
<referimentoCliente>
  <partitaIVA>01487450494</partitaIVA>
</referimentoCliente>
```

ma non è valido il seguente elemento XML

```
<referimentoCliente>
  <codiceFiscale>MNEGRG65C23E625Y</codiceFiscale>
  <partitaIVA>01487450494</partitaIVA>
</referimentoCliente>
```


perché l'indicatore `<xs:choice>` impone di scegliere un solo elemento tra quelli indicati.

Tutti gli indicatori presi in esame consentono una sola occorrenza di ogni singolo elemento: gli attributi `minOccurs` e `maxOccurs` permettono di indicare il numero minimo e massimo di occorrenze di ogni elemento. Lo speciale valore `unbounded` per l'attributo `maxOccurs` specifica l'assenza di un limite al numero di occorrenze dell'elemento.

ESEMPIO

Data la seguente definizione dell'elemento complesso `persona`

```
<xs:element name="persona">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="cognome" type="xs:string"/>
      <xs:element name="nome" type="xs:string"/>
      <xs:element name="figlio" type="xs:string"
        minOccurs="0" maxOccurs="10"/>
    </xs:sequence>
  </xs:complexType>
</xs:element>
```

sono validi i seguenti elementi XML:

```
<persona>
  <cognome>Meini</cognome>
  <nome>Giorgio</nome>
</persona>

<persona>
  <cognome>Meini</cognome>
  <nome>Giorgio</nome>
  <figlio>Federico</figlio>
  <figlio>Alessandro</figlio>
</persona>
```

Schemi XML estendibili

La definizione di un elemento di tipo `xs:any` nello schema XSD valida un elemento di tipo qualsiasi nel documento XML corrispondente (in particolare se viene impostato l'attributo `minOccurs` al valore 0 l'elemento diviene, oltre che generico, facoltativo).

Il tipo `xs:any` viene utilizzato per creare schemi XML flessibili che validano documenti con elementi non previsti in modo analogo per gli attributi si ricorre a `xs:anyAttribute`.

2.3 Tipi di dato predefiniti

La TABELLA 7 riassume alcuni tipi di dato predefiniti utilizzabili in uno schema XSD.

TABELLA 7

<code>xs:string</code>	Stringhe di caratteri.
<code>xs:normalizedString</code>	Stringhe di caratteri non suddivise su più linee e prive di tabulazioni.
<code>xs:token</code>	Stringhe di caratteri non suddivise su più linee, prive di tabulazioni, di spazi iniziali, finali e ripetuti.
<code>xs:float</code>	Valori numerici <i>floating-point</i> a 32 bit.
<code>xs:double</code>	Valori numerici <i>floating-point</i> a 64 bit.
<code>xs:decimal</code>	Valori numerici non necessariamente interi.
<code>xs:byte</code> <code>xs:unsignedByte</code>	Valori numerici rappresentabili con 8 bit, rispettivamente con segno o senza segno.

xs:short xs:unsignedShort	Valori numerici rappresentabili con 16 bit, rispettivamente con segno o senza segno.
xs:int xs:unsignedInt	Valori numerici rappresentabili con 32 bit, rispettivamente con segno o senza segno.
xs:long xs:unsignedLong	Valori numerici rappresentabili con 64 bit, rispettivamente con segno o senza segno.
xs:integer	Valori numerici interi.
xs:nonNegativeInteger	Valori numerici interi non negativi (compreso lo 0).
xs:negativeInteger	Valori numerici interi negativi.
xs:nonPositiveInteger	Valori numerici interi non positivi (compreso lo 0).
xs:positiveInteger	Valori numerici interi positivi.
xs:date	Data nel formato aaaa-mm-gg (a: anno, m: mese, g: giorno); un carattere «Z» finale opzionale indica che si tratta della data UTC.
xs:time	Ora nel formato oo:mm:ss (o: ora, m: minuti, s: secondi); i secondi possono assumere un valore non intero; un carattere «Z» finale opzionale indica che si tratta dell'ora UTC.
xs:dateTime	Data/ora nel formato aaaa-mm-ggT00:mm:ss (a: anno, m: mese, g: giorno, o: ora, m: minuti, s: secondi); i secondi possono assumere un valore non intero; un carattere «Z» finale opzionale indica che si tratta dell'ora UTC.
xs:duration	Durata temporale nel formato PaYmMgDTtoHmMsS (a: anni, m: mesi, g: giorni, o: ore, m: minuti, s: secondi; i secondi possono assumere un valore non intero); le varie sezioni sono opzionali, il carattere «P» inizia obbligatoriamente la rappresentazione del periodo, il carattere «T» inizia obbligatoriamente la parte oraria del periodo.
xs:boolean	Valori true/false .
xs:hexBinary	Sequenze di byte codificate in esadecimale.
xs:base64Binary	Sequenze di byte codificate in base-64.
xs:anyURI	Stringa che rappresenta un URI (<i>Uniform Resource Identifier</i>).

OSSERVAZIONI Relativamente ai tipi di data e ora, la data/ora UTC (*Universal Time Coordinated*) è la data ora riferita al meridiano di Greenwich (Londra).

Ai tipi di data, ora e durata è possibile applicare le restrizioni per i valori enumerati, minimi e massimi.

4. Base-64 è una codifica che impiega un sottoinsieme di 64 caratteri ASCII per realizzare un sistema di numerazione in base 64 spesso utilizzato per la rappresentazione testuale del contenuto di file binari.

ESEMPIO Data la seguente definizione dell'elemento complesso viaggio

```

<xs:element name="viaggio">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="mezzo">
        <xs:simpleType>
          <xs:restriction base="xs:string">
            <xs:enumeration value="auto"/>
            <xs:enumeration value="treno"/>
            <xs:enumeration value="aereo"/>
          </xs:restriction>
        </xs:simpleType>
      </xs:element>
      <xs:element name="partenza" type="xs:dateTime"/>
      <xs:element name="luogoPartenza" type="xs:string"/>
    </xs:sequence>
  </xs:complexType>
</xs:element>

```



```

<xs:element name="arrivo" type="xs:dateTime"/>
<xs:element name="luogoArrivo" type="xs:string"/>
<xs:element name="durata" type="xs:duration"/>
</xs:sequence>
</xs:complexType>
</xs:element>

```

è valido il seguente elemento XML:

```

<viaggio>
  <mezzo>treno</mezzo>
  <partenza>2021-10-21T10:35:10</partenza>
  <luogoPartenza>Livorno</luogoPartenza>
  <arrivo>2021-10-21T14:40:30</arrivo>
  <luogoArrivo>Milano</luogoArrivo>
  <durata>PT4H5M20S</durata>
</viaggio>

```

ESEMPIO

Data la seguente definizione dell'elemento complesso riferimenti

```

<xs:element name="riferimenti">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="riferimento" type="xs:anyURI"
        minOccurs="0" maxOccurs="unbounded"/>
    </xs:sequence>
  </xs:complexType>
</xs:element>

```

risultano validi i seguenti elementi XML:

```

<riferimenti>
  <riferimento>http://www.w3schools.com</riferimento>
  <riferimento>http://www.w3.org/XML/</riferimento>
</riferimenti>

<riferimenti>
</riferimenti>

```

2.4 Un esempio completo

Il percorso di un veicolo sulla superficie terrestre è una lista ordinata di posizioni geografiche – latitudine e longitudine espresse come gradi, minuti e secondi – associata alla data/ora in cui il veicolo le ha attraversate.

Il seguente schema XSD, memorizzato nel file percorso.xsd, definisce la struttura di un documento XML adatto a memorizzare il percorso di un veicolo:



```

<?xml version="1.0" ?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <!-- definizione del tipo latitudine -->
  <xs:complexType name="tipo_latitudine">
    <xs:sequence>
      <xs:element name="gradi">
        <xs:simpleType>

```

```

        <xs:restriction base="xs:integer">
            <xs:minInclusive value="0"/>
            <xs:maxExclusive value="90"/>
        </xs:restriction>
    </xs:simpleType>
</xs:element>
<xs:element name="minuti">
    <xs:simpleType>
        <xs:restriction base="xs:integer">
            <xs:minInclusive value="0"/>
            <xs:maxExclusive value="60"/>
        </xs:restriction>
    </xs:simpleType>
</xs:element>
<xs:element name="secondi">
    <xs:simpleType>
        <xs:restriction base="xs:decimal">
            <xs:minInclusive value="0"/>
            <xs:maxExclusive value="60"/>
        </xs:restriction>
    </xs:simpleType>
</xs:element>
<xs:element name="orientamento">
    <xs:simpleType>
        <xs:restriction base="xs:string">
            <xs:enumeration value="Nord"/>
            <xs:enumeration value="Sud"/>
        </xs:restriction>
    </xs:simpleType>
</xs:element>
</xs:sequence>
</xs:complexType>
<!-- definizione del tipo longitudine -->
<xs:complexType name="tipo_longitudine">
    <xs:sequence>
        <xs:element name="gradi">
            <xs:simpleType>
                <xs:restriction base="xs:integer">
                    <xs:minInclusive value="0"/>
                    <xs:maxExclusive value="180"/>
                </xs:restriction>
            </xs:simpleType>
        </xs:element>
        <xs:element name="minuti">
            <xs:simpleType>
                <xs:restriction base="xs:integer">
                    <xs:minInclusive value="0"/>
                    <xs:maxExclusive value="60"/>
                </xs:restriction>
            </xs:simpleType>
        </xs:element>
        <xs:element name="secondi">
            <xs:simpleType>
                <xs:restriction base="xs:decimal">

```



```

        <xs:minInclusive value="0"/>
        <xs:maxExclusive value="60"/>
      </xs:restriction>
    </xs:simpleType>
  </xs:element>
  <xs:element name="orientamento">
    <xs:simpleType>
      <xs:restriction base="xs:string">
        <xs:enumeration value="Est"/>
        <xs:enumeration value="Ovest"/>
      </xs:restriction>
    </xs:simpleType>
  </xs:element>
</xs:sequence>
</xs:complexType>
<!-- definizione del tipo posizione -->
<xs:complexType name="tipo_posizione">
  <xs:sequence>
    <xs:element name="latitudine" type="tipo_latitudine"/>
    <xs:element name="longitudine" type="tipo_longitudine"/>
    <xs:element name="dataOra" type="xs:dateTime"/>
  </xs:sequence>
</xs:complexType>
<!-- definizione dello schema percorso -->
<xs:element name="percorso">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="posizione" type="tipo_posizione"
        minOccurs="0" maxOccurs="unbounded"/>
    </xs:sequence>
  </xs:complexType>
</xs:element>
</xs:schema>

```

L'elemento radice percorso è composto da una lista potenzialmente infinita di elementi posizione di tipo tipo_posizione. Il tipo complesso tipo_posizione è costituito da un elemento latitudine di tipo tipo_latitudine, da un elemento longitudine di tipo tipo_longitudine e da un elemento dataOra di tipo predefinito dateTime. Entrambi i tipi complessi tipo_latitudine e tipo_longitudine sono a loro volta costituiti dagli elementi gradi, minuti, secondi e orientamento definiti come opportune restrizioni di tipi predefiniti. Si noti come il ricorso alla definizione di tipi renda – nonostante la verbosità tipica di XSD – lo schema leggibile.



ESEMPIO

Il seguente documento XML risulta valido rispetto allo schema precedente:

```

<?xml version="1.0" ?>
<percorso
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:noNamespaceSchemaLocation="percorso.xsd">
  <posizione>
    <latitudine>
      <gradi>43</gradi>

```

```

    <minuti>33</minuti>
    <secondi>0</secondi>
    <orientamento>Nord</orientamento>
  </latitudine>
  <longitudine>
    <gradi>10</gradi>
    <minuti>19</minuti>
    <secondi>0</secondi>
    <orientamento>Est</orientamento>
  </longitudine>
  <dataOra>2016-10-21T17:29:59</dataOra>
</posizione>
<posizione>
  <latitudine>
    <gradi>43</gradi>
    <minuti>43</minuti>
    <secondi>0</secondi>
    <orientamento>Nord</orientamento>
  </latitudine>
  <longitudine>
    <gradi>10</gradi>
    <minuti>24</minuti>
    <secondi>0</secondi>
    <orientamento>Est</orientamento>
  </longitudine>
  <dataOra>2016-10-21T17:59:09</dataOra>
</posizione>
</percorso>

```

3 API per la gestione di documenti XML con il linguaggio Java

Il linguaggio di programmazione Java prevede un'API (*Application Program Interface*) standard per la gestione di documenti XML denominata JAXP (*Java API for XML Processing*).

Tra le elaborazioni che le classi dei package che costituiscono JAXP consentono di effettuare sono fondamentali le seguenti:

- validazione di un documento XML rispetto a uno schema XSD;
- produzione di un nuovo documento XML ed eventuale salvataggio in un file di testo;
- lettura di un documento XML contenuto in un file di testo o reso disponibile come risorsa in rete ed eventuale verifica della correttezza;
- trasformazione di un documento XML.

La lettura di un documento XML in una struttura dati che è possibile gestire da parte del codice di un programma è tecnicamente definita *parsing*.

XML (EXTENSIBLE MARKUP LANGUAGE). XML è un metalinguaggio per la definizione di linguaggi basati su *tag* marcatori; la versatilità di XML lo ha reso uno strumento utilizzato principalmente per definire documenti con marcatori in cui i *tag* non sono predefiniti e collezioni di dati semi-strutturati rappresentate in formato testuale.

STRUTTURA AD ALBERO DI UN DOCUMENTO XML.

Dato che le regole per la nidificazione dei *tag* del linguaggio sono rigorosamente gerarchiche, i documenti in formato XML hanno naturalmente una struttura ad albero che viene utilizzata da molti strumenti che ne consentono la gestione.

ELEMENTI XML. Un elemento di un documento XML è quanto è compreso tra un *tag* di apertura e il corrispondente *tag* di chiusura; un elemento può contenere testo, attributi e altri elementi, oppure essere vuoto.

DOCUMENTI XML BEN FORMATI. Un documento XML che rispetta le regole sintattiche di base è denominato ben formato.

XSD (XML SCHEMA DEFINITION). Uno schema XSD consente di definire la struttura e i tipi degli elementi che un documento XML deve rispettare per essere considerato valido. La validazione di un documento XML rispetto a uno schema XSD viene effettuata mediante strumenti software automatici; ogni documento XML dovrebbe contenere un riferimento allo schema XSD che lo definisce.

TIPI XSD PREDEFINITI E DERIVATI. In uno schema XSD è possibile imporre un tipo al testo contenuto in un elemento. I tipi predefiniti fondamentali sono le stringhe di caratteri, i valori numerici generali, i valori numerici interi, le date e gli orari; a questi tipi fondamentali è possibile imporre delle restrizioni, per esempio una limitazione del campo di validità per i tipi numerici. In uno schema XSD è possibile definire nuovi tipi a partire dai tipi fondamentali, dalle loro restrizioni e dalle loro combinazioni.

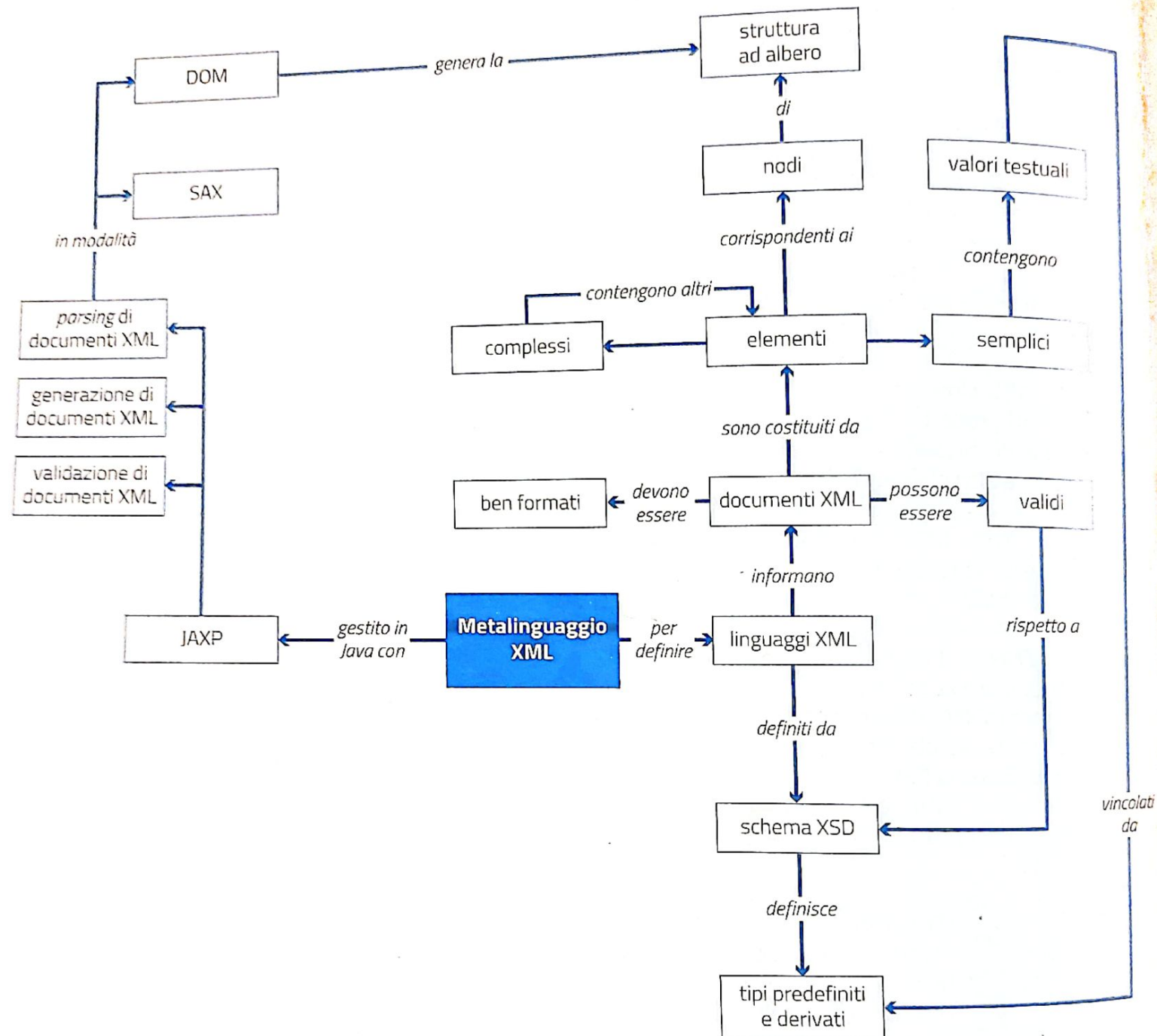
ELEMENTI XML SEMPLICI E COMPLESSI. Un elemento XML semplice è un elemento che può contenere solo testo. Un elemento XML complesso è normalmente un elemento che contiene attributi e/o altri elementi; gli indicatori XSD consentono di definire un elemento complesso come sequenza di più elementi, anche complessi.

XML E JAVA. Il linguaggio di programmazione Java prevede un'API (*Application Program Interface*) standard per la gestione di documenti XML denominata JAXP (*Java API for XML Processing*). Tra le elaborazioni che le classi dei package che costituiscono JAXP consentono di effettuare sono fondamentali le seguenti: validazione di un documento XML rispetto a uno schema XSD, produzione di un nuovo documento XML ed eventuale salvataggio in un file di testo, lettura di un documento XML contenuto in un file di testo o reso disponibile come risorsa in rete, trasformazione di un documento XML. Molte classi JAXP fondamentali devono essere istanziate seguendo la logica del *design pattern Factory-method*: l'invocazione di specifici metodi della classe *Factory* restituisce i riferimenti alle classi istanziate che sono successivamente utilizzate per operare.

PARSING DI DOCUMENTI XML. La lettura di un documento XML in una struttura dati che è possibile gestire da parte del codice di un programma è tecnicamente definita *parsing*. Tradizionalmente il *parsing* di un documento XML da parte di un programma si basa su una delle due seguenti modalità fondamentali: SAX (*Simple API for XML*) e DOM (*Document Object Model*). I vari package che costituiscono JAXP permettono di effettuare il *parsing* con modalità SAX, DOM e StAX.

PARSING DOM. Nella modalità di *parsing* DOM il documento viene interamente mappato in una struttura ad albero che rappresenta la struttura gerarchica degli elementi del documento XML e il codice può liberamente navigare i nodi dell'albero. La modalità DOM consente di modificare il documento XML originale e anche di crearne dinamicamente uno completamente nuovo.

RIPASSA CON LA MAPPA



1 XML è...

- ☐ A un linguaggio con *tag* marcatori alternativo rispetto ad HTML.
- ☐ B un metalinguaggio per la definizione di linguaggi con *tag* marcatori.
- ☐ C un metalinguaggio da cui è stato derivato HTML.
- ☐ D un linguaggio derivato da HTML.

2 I *tag* di un documento XML...

- ☐ A possono essere aperti e non chiusi.
- ☐ B devono essere nidificati in modo gerarchico.
- ☐ C devono necessariamente prevedere uno o più attributi.
- ☐ D possono essere indifferentemente codificati con caratteri minuscoli o maiuscoli.

3 Indicare che cosa può contenere un elemento XML.

- ☐ A Valori testuali.
- ☐ B Uno o più attributi.
- ☐ C Altri elementi.
- ☐ D Commenti.

4 Un documento XML può essere rappresentato mediante...

- ☐ A un albero.
- ☐ B una lista.
- ☐ C una tabella.
- ☐ D un vettore.

5 Quale scopo ha la specificazione dell'*encoding* di un documento XML?

- ☐ A Consentire la corretta interpretazione dei caratteri che costituiscono il documento.
- ☐ B Specificare il metodo di compressione del documento.
- ☐ C Indicare il metodo di cifratura del documento.
- ☐ D Individuare il formato del file che contiene il documento.

6 Per quale motivo si usa un elemento di tipo *CDATA* in un documento XML?

- ☐ A Per evitare che il contenuto dell'elemento sia analizzato.
- ☐ B Per forzare l'interpretazione del contenuto dell'elemento.

- ☐ C Per inserire come contenuto dell'elemento una struttura dati definita in linguaggio C.
- ☐ D Per inserire nel documento un commento.

7 Un documento XML ben formato è un documento XML...

- ☐ A che rispetta le regole sintattiche del linguaggio.
- ☐ B validato rispetto a uno schema XSD.
- ☐ C validato rispetto a uno schema pubblicato sul web.
- ☐ D correttamente indentato.

8 Un documento XML valido è un documento XML...

- ☐ A che rispetta le regole sintattiche del linguaggio.
- ☐ B validato rispetto a uno schema XSD.
- ☐ C utilizzato da più di un'applicazione software.
- ☐ D validato rispetto a uno schema pubblicato sul web.

9 XSD è un linguaggio...

- ☐ A alternativo a XML.
- ☐ B XML per la definizione di schemi per la validazione di documenti XML.
- ☐ C per la trasformazione e la visualizzazione di documenti XML.
- ☐ D non XML per la definizione di schemi per la validazione di documenti XML.

10 Uno schema XSD definisce il tipo...

- ☐ A di tutti gli elementi testuali di un documento XML.
- ☐ B dei soli valori degli attributi di un documento XML.
- ☐ C dei soli elementi testuali con formato numerico di un documento XML.
- ☐ D dei soli elementi testuali con formato non numerico di un documento XML.

11 La restrizione del tipo di un elemento XML in uno schema XSD...

- ☐ A pone ulteriori condizioni alla validità di un elemento testuale di un determinato tipo.
- ☐ B è applicabile esclusivamente a elementi testuali di tipo numerico.
- ☐ C è applicabile esclusivamente a elementi testuali di tipo stringa di caratteri.
- ☐ D rilassa alcune condizioni alla validità di un elemento testuale di un determinato tipo.

12 Un elemento XML complesso è un elemento XML che...

- A** contiene esclusivamente testo.
- B** può contenere altri elementi.
- C** contiene necessariamente attributi.
- D** non può contenere altri elementi.

13 L'indicatore `<xs:sequence>` in uno schema XSD consente di definire un elemento...

- A** complesso con più elementi nell'ordine specificato.
- B** semplice il cui contenuto è di tipo sequenziale.
- C** complesso con più elementi in cui l'ordine non ha importanza.
- D** complesso costituito unicamente dalla ripetizione di uno stesso elemento.

14 Dovendo definire un elemento che specifica una data è...

- A** necessario definire un elemento complesso che contiene gli elementi giorno, mese e anno.
- B** sufficiente definire un elemento semplice del tipo predefinito `date`.
- C** necessario definire un elemento complesso del tipo predefinito `date`.
- D** necessario definire un elemento semplice del tipo predefinito `dateTime`.

15 Quali elaborazioni consentono di effettuare le classi dei package che costituiscono JAXP?

- A** Validazione di un documento XML rispetto a uno schema XSD.
- B** Generazione di un nuovo documento XML.
- C** Parsing di un documento XML ed eventuale verifica della correttezza.
- D** Trasformazione di un documento XML.

16 Un parser SAX...

- A** costruisce l'intero albero del documento XML.
- B** analizza il documento XML generando specifici eventi in corrispondenza di determinate tipologie di elementi.
- C** permette di modificare un documento XML.
- D** permette di navigare un documento XML.

17 Un parser DOM...

- A** costruisce l'intero albero del documento XML.

- B** analizza il documento XML generando specifici eventi in corrispondenza di determinate tipologie di elementi.
- C** permette di navigare un documento XML.
- D** non permette di modificare un documento XML.

PROBLEMI

18 Indicare se i seguenti documenti XML sono ben formati. In caso contrario specificare il motivo per cui non lo sono.

A. `<?xml version="1.0" ?>`

```
<A>
  <B>...</B>
  <C>...</C>
</A>
```

`<A>`

...

`</A>`

B. `<?xml version="1.0" ?>`

```
<A>
  <B>...</C>
  <C>...</B>
</A>
```

C. `<?xml version="1.0" ?>`

```
<A>
  <B>...</b>
  <c>...</C>
</A>
```

D. `<?xml version="1.0" ?>`

```
<A x=123>
  <B>...</B>
  <C>...</C>
</A>
```

E. `<?xml version="1.0" ?>`

```
<A y="123">
  <B>...</C>
  <C>...</B>
</A>
```

CASI PRATICI

19 **DESIGN** Progettare mediante uno schema XSD un linguaggio XML che consenta di rappresentare una rubrica dei riferimenti a persone

(nome, cognome, indirizzo, uno o più numeri di telefono, uno o più indirizzi di posta elettronica, sito web). Produrre un documento XML valido di esempio.

20 DESIGN Progettare mediante uno schema XSD un linguaggio XML per la creazione di vocabolari intesi come elenchi di termini e in cui per ogni termine siano specificati uno o più significati. Produrre un documento XML valido di esempio.

21 DESIGN Progettare mediante uno schema XSD un linguaggio XML per la rappresentazione di un giornale composto da articoli; ogni articolo ha un titolo, un sottotitolo, un'intestazione, un autore, una data e un corpo composto da uno o più paragrafi di testo. Produrre un documento XML valido di esempio.

22 DESIGN Progettare mediante uno schema XSD un linguaggio XML per la realizzazione di registri dell'insegnante in cui per ogni studente sono riportati il nome, il cognome, la classe, le date dei giorni di assenza, le valutazioni comprendenti la data, la tipologia (scritta, orale, pratica, test), il punteggio e il voto decimale. Produrre un documento XML valido di esempio.

23 DESIGN Progettare mediante uno schema XSD un linguaggio XML per la definizione di prove di verifica strutturate composte da quesiti a scelta multipla; per ogni quesito è necessario specificare la richiesta, il punteggio e le possibili risposte. Produrre un documento XML valido di esempio.

24 DESIGN Definire mediante uno schema XSD un linguaggio XML che consenta di rappresentare le attività da effettuare per completare uno o più progetti. Per ogni attività devono essere riportate le seguenti informazioni:

- descrizione;
- data di scadenza;
- percentuale di avanzamento;
- priorità (ALTA, MEDIA, BASSA).

Un documento XML conforme al linguaggio può contenere le attività relative a più progetti; per ogni progetto – oltre all'elenco delle attività – sono riportate le seguenti informazioni:

- denominazione;
- elenco delle persone coinvolte (cognome e nome);

- budget disponibile (€).

Progettare mediante uno schema XSD il linguaggio XML definito e produrre almeno un documento XML valido di esempio.

25 DESIGN Una piattaforma di streaming necessita di associare ai video che importa e diffonde dei metadati di classificazione.

A questo scopo si procede a definire un linguaggio XML che per ogni video preveda:

- il titolo del video e l'eventuale categoria;
- una sintetica descrizione testuale del video;
- la tipologia di video (film, notiziario, serie, animazione, documentario, evento);
- la durata del video;
- la lingua dell'audio del video;
- la risoluzione del video (720p, 1080p, 1080i, 4K);
- l'eventuale numero di *like* e di *unlike*;
- la data/ora di caricamento del video sulla sorgente;
- l'elenco dei *tag* di classificazione del video; per ogni *tag* è necessario specificare:
 - il contenuto testuale del *tag*;
 - la data/ora di generazione del *tag*;
- la descrizione delle sezioni in cui è suddiviso il video riportando per ciascuna il tempo di inizio e di fine rispetto all'inizio del video (può trattarsi di una sola sezione).

Progettare mediante uno schema XSD il linguaggio XML definito e produrre almeno un documento XML valido di esempio.

26 DESIGN Un sito di cucina intende rinnovare il formato delle proprie ricette. A questo scopo si procede a definire un linguaggio XML che per una ricetta preveda:

- il titolo della ricetta;
- la categoria della ricetta (antipasti, primi piatti, secondi piatti, contorni, dessert);
- il numero di porzioni cui si riferiscono le dosi indicate negli ingredienti;
- la spesa indicativa per l'acquisto degli ingredienti;
- l'elenco degli ingredienti; per ogni ingrediente è necessario specificare:
 - il nome dell'ingrediente;
 - la quantità dell'ingrediente con eventuale indicazione dell'unità di misura (grammi, cucchiaini, ...);

- la descrizione della preparazione e della cottura suddivisa in fasi; per ciascuna fase deve essere riportato:
 - il numero progressivo di esecuzione;
 - la denominazione;
 - la descrizione delle attività di preparazione o cottura;
 - la durata temporale.

Il rinnovamento del sito prevede inoltre che gli utenti registrati – identificati da un username – possano esprimere una valutazione sulle ricette che comprende un voto da 1 a 5 e un breve commento.

Progettare mediante uno schema XSD il linguaggio XML definito e produrre almeno un documento XML valido di esempio.

27 DESIGN I pazienti in riabilitazione cardiologica sono dotati di un dispositivo holter che registra a intervalli non regolari di tempo i seguenti parametri fisiologici associandoli alla data/ora di registrazione:

- stato di allarme ("---", "F--", "-P-", "--O", "FP-", "F-O", "-PO", "FPO");
- frequenza cardiaca compresa tra 0 e 300 Hz;
- pressione sistolica compresa tra 0 e 250 mmHg;
- pressione diastolica compresa tra 0 e 250 mmHg;
- percentuale di ossigenazione del sangue;
- 10 secondi di cardiogramma costituito da 100 valori compresi tra -5 e +5 V.

Al termine della sessione di riabilitazione il dispositivo holter viene connesso a un PC per l'esportazione di un file XML comprendente l'identificativo numerico del dispositivo costituito da un codice alfanumerico di 10 caratteri.

Progettare il linguaggio XML di esportazione dei dati del dispositivo holter mediante uno schema XSD.

28 DESIGN Con riferimento all'esercizio precedente, per ogni paziente viene predisposto dal medico un file XML che, caricato sul dispositivo holter, specifica i valori minimi e massimi per ogni parametro fisiologico (frequenza cardiaca, pressione sistolica, pressione diastolica, ossigenazione del sangue): il dispositivo utilizza i valori presenti in questo file per la generazione degli stati di allarme. Progettare il linguaggio XML del file dei valori minimi/massimi dei parametri fisiologici mediante uno schema XSD.

29 DESIGN Progettare mediante uno schema XSD un linguaggio XML che consenta di rappresentare le case vendute su un sito web specializzato. Per ogni casa il sito visualizza le seguenti informazioni:

- tipologia (appartamento, villetta, villa, rustico, colonica);
- indirizzo (città, via, numero civico);
- piano;
- ascensore (sì/no);
- descrizione testuale;
- categoria energetica (A, B, C, D, E o F);
- numero di vani;
- superficie (m^2);
- costo (€).

Prevedere le necessarie restrizioni/estensioni dei tipi di base e produrre almeno un esempio di documento XML valido secondo lo schema fornito. Estendere successivamente lo schema XSD in modo che a ogni casa in vendita sia associato l'elenco delle agenzie immobiliari che ne dispongono la vendita: per ogni agenzia devono essere specificati la denominazione, l'indirizzo, il numero di telefono, l'indirizzo di posta elettronica e l'eventuale sito web.

PROGETTI

30 DESIGN CODING Progettare mediante uno schema XSD un linguaggio XML che consenta di rappresentare i cassonetti della spazzatura presenti in una città. Per ogni cassonetto è necessario disporre delle seguenti informazioni:

- tipologia (generico, organico, carta, vetro e metallo, tessuti);
- indirizzo (via e numero civico);
- coordinate geografiche (latitudine e longitudine espresse come valori decimali);
- note testuali;
- data di posizionamento;
- data/ora di ultimo svuotamento;
- volume (m^3).

Prevedere le necessarie restrizioni/estensioni dei tipi di base e produrre almeno un esempio di documento XML valido secondo lo schema fornito.

Estendere successivamente lo schema XSD in modo che per ogni cassonetto sia presente l'e-

lenco degli operatori e dei veicoli che hanno operato lo svuotamento: per ogni operazione di svuotamento devono essere specificate la matricola dell'addetto, la targa del veicolo e la data/ora dell'operazione.

Realizzare un programma in linguaggio Java che, dopo aver validato un documento XML contenuto in un file il cui nome è fornito come argomento della riga di comando, rispetto allo schema XSD progettato, produca l'elenco dei cassonetti di categoria «organico» non svuotati nelle ultime 24 ore.

31 **DESIGN CODING** Progettare mediante uno schema XSD un linguaggio XML che consenta di rappresentare i modelli di auto disponibili per il noleggio mediante un sito web. Per ogni auto il sito visualizza le seguenti informazioni:

- classe (A, B, C, D o E);
- marca;
- modello;
- cilindrata (cm³);
- potenza (KW);
- alimentazione (benzina, GPL, metano, diesel, elettrica o ibrida);
- numero passeggeri;
- volume bagagliaio (l).

Prevedere le necessarie restrizioni/estensioni dei tipi di base e produrre almeno un esempio di documento XML valido secondo lo schema fornito. Estendere successivamente lo schema XSD in modo che a ogni modello di auto sia associato l'elenco delle agenzie in cui è possibile noleggiarlo: per ogni agenzia devono essere specificate la denominazione e la città sotto forma di attributi.

Realizzare un programma in linguaggio Java che, dopo aver validato un documento XML contenuto in un file il cui nome è fornito come argomento della riga di comando, rispetto allo schema XSD progettato, produca l'elenco delle agenzie in cui è possibile noleggiare un'auto ad alimentazione «ibrida» di una classe specificata come ulteriore argomento della riga di comando.

32 **DESIGN CODING** Il percorso di volo di un drone viene memorizzato in un documento in formato XML come il seguente dove sono memorizzate le coordinate geografiche e l'altitudine associate a istanti temporali successivi:

```
<percorso>
  <posizione>
```

```
    <latitudine>43.55</latitudine>
    <longitudine>10.3167</longitudine>
    <altitudine>100.9</altitudine>
    <dataOra>2022-10-21T17:29:59</dataOra>
  </posizione>
  <posizione>
    <latitudine>43.65</latitudine>
    <longitudine>10.3333</longitudine>
    <altitudine>99.1</altitudine>
    <dataOra>2022-10-21T17:39:47</dataOra>
  </posizione>
  <posizione>
    <latitudine>43.70</latitudine>
    <longitudine>10.37</longitudine>
    <altitudine>101.2</altitudine>
    <dataOra>2022-10-21T17:49:07</dataOra>
  </posizione>
  <posizione>
    <latitudine>43.7167</latitudine>
    <longitudine>10.4</longitudine>
    <altitudine>98.9</altitudine>
    <dataOra>2022-10-21T17:59:09</dataOra>
  </posizione>
</percorso>
```

Dopo aver definito una classe Java Posizione che permette di rappresentare un elemento posizione del documento XML, codificare una classe in linguaggio Java i cui metodi consentano di:

- inserire in un array (o un ArrayList) le successive posizioni del drone riportate in un file il cui contenuto è un documento XML con il formato illustrato;
- determinare la latitudine e la longitudine massime e minime del percorso effettuato dal drone.

33 **DESIGN CODING** Il percorso di una spedizione presa in carico da un corriere viene memorizzato in un documento in formato XML come il seguente dove sono memorizzati le località di partenza, transito e di arrivo associate alla relativa data/ora di registrazione:

```
<spedizione>
  <codice>0123456789</codice>
  <località>
    <luogo>Pisa, aeroporto</luogo>
    <data-ora>2022-03-7T12:00:15</data-ora>
  </località>
  <località>
    <luogo>Pisa, stazione</luogo>
```

```

<data-ora>2022-03-7T18:00:30</data-ora>
</località>
<località>
  <luogo>Livorno, stazione</luogo>
  <data-ora>2022-03-7T22:15:45</data-ora>
</località>
</spedizione>

```

Dopo aver definito una classe Java *Località* che permetta di rappresentare un elemento località del documento XML, codificare una classe in linguaggio Java i cui metodi consentano di:

- inserire in un array (o un *ArrayList*) le successive località attraversate da una spedizione

ne riportate in un file il cui contenuto è un documento XML con il formato illustrato;

- determinare il tempo trascorso tra la partenza (prima località) e l'arrivo (ultima località) della spedizione.

Integrare successivamente la classe realizzata di un metodo che produca un file contenente un documento XML riportante i tempi espressi in secondi tra una località e la successiva:

```

<spedizione>
  <tempo>21615</tempo>
  <tempo>15315</tempo>
</spedizione>

```

34 **DESIGN** **CODING** I veicoli per il trasporto di prodotti congelati sono dotati di un termometro che rileva periodicamente la temperatura del vano di carico; quando giungono a destinazione un operatore dotato di un dispositivo mobile controlla le misure di temperatura rilevate da parte dei termometri di uno o più veicoli. L'APP del dispositivo mobile produce, a partire dalle misure, un documento XML strutturato come il seguente:

```

<veicoli>
  <veicolo>
    <ID>1234567890</ID>
    <misura>
      <temperatura>-31.2</temperatura>
      <data_ora>2022-03-10T00:31:59</data_ora>
    </misura>
    <misura>
      <temperatura>-30.9</temperatura>
      <data_ora>2022-03-10T01:01:22</data_ora>
    </misura>
  </veicolo>
  <veicolo>
    <ID>9876543210</ID>
    <misura>
      <temperatura>-29.9</temperatura>
      <data_ora>2022-03-10T05:55:59</data_ora>
    </misura>
    <misura>
      <temperatura>-30.5</temperatura>
      <data_ora>2022-03-10T06:31:19</data_ora>
    </misura>
    <misura>
      <temperatura>-30.9</temperatura>
      <data_ora>2022-03-10T07:11:01</data_ora>
    </misura>
  </veicolo>
</veicoli>

```


È richiesta un'applicazione in linguaggio Java che, a partire da un documento XML come il precedente e da un valore di soglia di temperatura, fornisca per ogni veicolo le seguenti informazioni:

- un valore booleano che indichi se tutti i valori misurati sono inferiori al valore di soglia, o se anche uno solo di essi è superiore;
- l'elenco dei *timestamp* (espressi come numero di secondi a partire dalle 00:00:00 del 1/1/1970) in cui la temperatura misurata è superiore al valore di soglia.

35 DESIGN CODING In una competizione scolastica di mini-robotica il giudice addetto a ogni campo di gara utilizza un'APP per smartphone/tablet per registrare il punteggio di ogni squadra al termine di ogni singola gara; l'APP produce un documento XML avente il seguente formato:

```
<gara>
  <campo>A</campo>
  <giudice>Robot</giudice>
  <squadra>Galilei</squadra>
  <ora>12:30:01</ora>
  <punteggio_percorso>99</punteggio_percorso>
  <punteggio_difficoltà>101</punteggio_difficoltà>
  <tempo>600</tempo>
</gara>
```

L'APP inoltra i documenti XML al server di calcolo dei punteggi dove sono salvati come file in directory separate in base alla squadra. È richiesta un'applicazione Java che, a partire da più file in formato XML relativi alla stessa squadra, determini il punteggio e il tempo complessivi come somma dei punteggi e dei tempi delle singole gare svolte in campi diversi.

Nell'ipotesi che i file contenenti i documenti XML delle diverse squadre siano salvati nella stessa directory modificare l'applicazione Java di calcolo del punteggio in modo che determini il punteggio di tutte le squadre.

36 DESIGN CODING Progettare mediante uno schema XSD un linguaggio XML per rappresentare i dati prodotti da una stazione meteorologica che, per motivi di risparmio energetico, nell'arco di una giornata effettua misure esclusivamente in alcuni intervalli di tempo predefiniti (fornire almeno un esempio di documento XML valido rispetto allo schema). Ogni documento deve contenere le seguenti informazioni:

- codice identificativo della stazione meteorologica;
- latitudine e longitudine di localizzazione della stazione meteorologica;
- data di rilevazione dei dati;
- elenco delle rilevazioni, ciascuna delle quali comprende:
 - ora di rilevazione;
 - temperatura compresa tra $-50,0^{\circ}\text{C}$ e $+50,0^{\circ}\text{C}$;
 - umidità compresa tra 0% e 100%;
 - pressione atmosferica compresa tra 700 MB e 1200 MB;
 - velocità del vento compresa tra 0 km/h e 500 km/h;
 - direzione del vento compresa tra 0° e 360° ;
 - pioggia caduta compresa tra 0 mm e 100 mm.

Realizzare un programma in linguaggio Java che, dopo aver validato un documento XML rispetto allo schema XSD progettato, ne visualizzi i valori medi per l'intera giornata. Il nome del documento XML deve essere fornito come argomento della riga di comando.

VERSO L'ESAME

Risolvi con l'aiuto del Manuale Cremonese di informatica e telecomunicazioni

37 **DESIGN CODING** Progettare mediante uno schema XSD un linguaggio XML per rappresentare gli esiti delle analisi del sangue di un paziente; fornire almeno un esempio di documento XML valido rispetto allo schema. Ogni esito deve contenere le seguenti informazioni:

- nome, cognome, codice fiscale, sesso ed età del paziente;
- data e ora del prelievo di sangue;
- luogo del prelievo (centro territoriale, reparto ospedaliero, domicilio del paziente, pronto soccorso o ambulanza);
- elenco degli esami; per ogni esame è necessario riportare:
 - data e ora di effettuazione dell'analisi;
 - codice dell'operatore che ha eseguito l'analisi (costituito da 9 caratteri numerici);
 - numeri di matricola degli strumenti utilizzati per l'analisi (massimo 5 strumenti, 0 strumenti in caso di analisi manuale; il numero di matricola è un numero intero positivo);
 - denominazione dell'analisi;
 - valore numerico e unità di misura del risultato;
 - valori minimi e massimi di riferimento per il risultato.

Progettare successivamente mediante uno schema XSD un secondo linguaggio XML i cui documenti consentano a un'applicazione software di verificare, a partire da un documento XML degli esiti delle analisi del sangue, che tutte le analisi sono state effettuate entro il periodo di tempo specificatamente previsto dal momento del prelievo, utilizzando strumenti abilitati all'analisi specifica da parte di operatori abilitati all'uso dello strumento per l'esecuzione dell'analisi (un operatore non è abilitato a eseguire una specifica analisi in generale, ma lo è a impiegare uno specifico strumento per eseguire una specifica analisi).

Realizzare un programma Java che implementi l'applicazione di verifica di un'analisi.



GUARDA sul
Manuale Cremonese
di informatica
e telecomunicazioni